

Two P1 bugs in the Daily Lot engine

Greptile's first automated review of the ProofLabAI codebase, run against pull request #1 on 23 August 2026. It found two defects. Both were real, both were reproduced before being fixed and again afterwards, and both are now live.

Reviewed	PR #1 — 23 files, +3,924 / -45 lines, three commits from 22–23 August
Findings	2 raised, 2 confirmed real, 2 fixed. Both rated P1 ; one also tagged Security.
Method	Greptile ran the code rather than only reading it, and attached the runs as evidence.
Fixed in	Commit ca6ce47, deployed to production the same day

P1

Paying the AI provider twice for the same work

supabase/functions/lot-writer/index.ts

What happens

A Lot — the day's piece of work — is written once per topic by an AI model and then shared by every student who reaches that topic. That design exists precisely to keep the bill small: 146 topics means at most 146 paid calls, ever.

The check for “has this topic been written yet?” and the write that records the answer were separated by the model call itself, which takes several seconds. Two students arriving inside that window both saw an unwritten topic and both paid for it.

```
student A  --read: not written--> call the model (5s)  --> save  PAID
student B  --read: not written--> call the model (5s)  --> save  PAID AGAIN
                                     ^
                               the two only converge after both bills
```

Why it matters

It is money, and it scales the wrong way. On the morning a cohort of students reaches the same new topic together, the number of duplicate charges rises with the number of students arriving at once — the opposite of what the write-once design was built to guarantee.

The fix

A claim, taken in the database before any money is spent. One caller wins a conditional update and generates; everyone else is told the topic is being written and returns immediately. The claim expires after two minutes, so a crashed or timed-out generation cannot lock a topic out permanently, and it is released explicitly on every failure path.

```
three simultaneous requests, one fresh topic:

caller 2  written: true, provider deepseek    <- one paid call
caller 0  "another request is writing this topic"
caller 1  "another request is writing this topic"
```

Verified against the live system with three concurrent HTTP calls under a real student session — not a simulation.

The Daily Card could fail to load, or stop the nightly run

supabase/migrations — create_lot_for()

What happens

The same shape of race, one layer down. The function asks whether the student already has a Lot for today; if not, it creates one. A database rule allows only one Lot per student per day. Two callers passing that check together both tried to insert, and the loser was handed a duplicate-key error rather than the Lot that had just been created.

```
caller A   no Lot today --> insert    OK
caller B   no Lot today --> insert    ERROR duplicate key

who hits this:  the nightly job and a student opening the page
                  at the same moment, or one page opened twice
```

Why it matters

Two different failures, depending on who loses. A student sees an empty Daily Card — the one screen the product opens on — with no explanation and no work to do. Or the nightly job that prepares tomorrow's Lots aborts mid-loop, and every student after that point in the run wakes up with nothing.

The fix

The insert now tolerates the collision instead of raising on it, and the caller that loses reads back the winner's Lot and reports it as today's Lot — which is exactly what it is.

two callers straddling the check:

```
caller A   created: true    task 5e885339
caller B   created: false   task 5e885339  <- same Lot, no error
rows for the day: 1
```

A third bug, found while fixing the first

The first deployment of the claim refused every caller, including the one that should have won, so nothing was generated at all. The cause is a trap worth recording, because it will recur:

```
revoke all on function ... from public, anon, authenticated

^ the standard sweep that keeps new functions away from browsers.
  It works because Postgres grants EXECUTE to PUBLIC by default -
  which means it also strips service_role, the identity the edge
  functions run as. Any function an edge function calls has to be
  granted back by name.
```

Caught by re-running the concurrency test after deploying, rather than by assuming the fix worked. It is now written into the migration so the next person meets the explanation before they meet the bug.

Status

Finding	Severity	State	Proof
Duplicate provider charges	P1 · Security	Fixed, live	3 concurrent calls, 1 paid
Daily Lot duplicate key	P1	Fixed, live	2 racing callers, 1 row
service_role lost EXECUTE	Found in-house	Fixed, live	Re-ran the same test

What this review does not cover

Both findings are races — two things happening at the same instant. That is the class of bug hardest to find by clicking through screens, and the class this review was worth running for. It says nothing about whether the features behave correctly for one user at a time; that was established separately by walking each flow against the live database.

Greptile reviews pull requests only. Work pushed straight to the main branch is never seen by it — which is why nothing before this pull request has been reviewed.